Lab Sheet-05 : Solutions

BCA201P - Foundations of Java Development

Encapsulation, Inheritance, Interfaces and Method Overriding

Name: _____ Roll No: _____

Q1. Student Encapsulation

Concept: private, getter, setter, data hiding

Program:

```
class Student {
    // private data members (data hiding)
    private String name;
    private int rollNo;
    private double marks;

    // getters and setters
    public String getName() { return name; }
    public void setName(String name) { this.name = name; }

    public int getRollNo() { return rollNo; }
    public void setRollNo(int rollNo) { this.rollNo = rollNo; }

    public double getMarks() { return marks; }
    public void setMarks(double marks) { this.marks = marks; }

    void displayDetails() {
        System.out.println("Name   : " + name);
        System.out.println("Roll No: " + rollNo);
        System.out.println("Marks  : " + marks);
    }
}

class Main {
    public static void main(String[] args) {
        Student s = new Student();

        // set values through setters
        s.setName("Varnika");
        s.setRollNo(101);
        s.setMarks(88.5);

        // display values through getters
        System.out.println("--- Using Getters ---");
        System.out.println("Name   : " + s.getName());
        System.out.println("Roll No: " + s.getRollNo());
        System.out.println("Marks  : " + s.getMarks());

        System.out.println("--- Using displayDetails() ---");
        s.displayDetails();
    }
}
```

OUTPUT:-

```
--- Using Getters ---
Name   : Varnika
Roll No: 101
Marks  : 88.5
--- Using displayDetails() ---
Name   : Varnika
Roll No: 101
Marks  : 88.5
```

Q2. Bank Account with Validation

Concept: encapsulation, controlled access, validation

Program:

```
class BankAccount {
    private String accountNumber;
```

```
    private String accountHolder;
    private double balance;

    public String getAccountNumber() { return accountNumber; }
    public void setAccountNumber(String accountNumber) { this.accountNumber = accountNumber; }

    public String getAccountHolder() { return accountHolder; }
    public void setAccountHolder(String accountHolder) { this.accountHolder = accountHolder; }

    public double getBalance() { return balance; }
    public void setBalance(double balance) {
        if (balance >= 0) this.balance = balance;
        else System.out.println("Invalid balance! Balance cannot be negative.");
    }

    void deposit(double amount) {
        if (amount > 0) {
            balance = balance + amount;
            System.out.println("Deposited " + amount + " | New Balance: " + balance);
        } else {
            System.out.println("Invalid deposit! Amount must be greater than 0.");
        }
    }

    void withdraw(double amount) {
        if (amount <= 0) {
            System.out.println("Invalid withdrawal! Amount must be greater than 0.");
        } else if (amount > balance) {
            System.out.println("Insufficient balance! Cannot withdraw " + amount
                    + " (Available: " + balance + ")");
        } else {
            balance = balance - amount;
            System.out.println("Withdrawn " + amount + " | New Balance: " + balance);
        }
    }
}

class Main {
    public static void main(String[] args) {
        BankAccount acc = new BankAccount();
        acc.setAccountNumber("1234567890");
        acc.setAccountHolder("Varnika");
        acc.setBalance(5000);

        System.out.println("Account No    : " + acc.getAccountNumber());
        System.out.println("Account Holder: " + acc.getAccountHolder());
        System.out.println("Balance       : " + acc.getBalance());
        System.out.println();

        acc.deposit(2000);     // valid
        acc.deposit(-500);     // invalid
        acc.withdraw(1500);    // valid
        acc.withdraw(10000);   // invalid (exceeds balance)
        acc.withdraw(0);       // invalid
    }
}
```

OUTPUT:-

```
Account No    : 1234567890
Account Holder: Varnika
Balance       : 5000.0

Deposited 2000.0 | New Balance: 7000.0
Invalid deposit! Amount must be greater than 0.
Withdrawn 1500.0 | New Balance: 5500.0
Insufficient balance! Cannot withdraw 10000.0 (Available: 5500.0)
Invalid withdrawal! Amount must be greater than 0.
```

Q3. Employee Salary Encapsulation

Concept: validation inside a setter

Program:

```
class Employee {
    private int employeeId;
```

```
    private String employeeName;
    private double salary;

    public int getEmployeeId() { return employeeId; }
    public void setEmployeeId(int employeeId) { this.employeeId = employeeId; }

    public String getEmployeeName() { return employeeName; }
    public void setEmployeeName(String employeeName) { this.employeeName = employeeName; }

    public double getSalary() { return salary; }

    // validation inside the setter
    public void setSalary(double salary) {
        if (salary < 0) {
            System.out.println("Rejected: Salary cannot be negative (" + salary + ")");
        } else if (salary > 1000000) {
            System.out.println("Rejected: Salary cannot exceed 1,000,000 (" + salary + ")");
        } else {
            this.salary = salary;
            System.out.println("Accepted: Salary set to " + salary);
        }
    }

    void displayDetails() {
        System.out.println("Employee ID  : " + employeeId);
        System.out.println("Employee Name: " + employeeName);
        System.out.println("Salary       : " + salary);
    }
}

class Main {
    public static void main(String[] args) {
        Employee e = new Employee();
        e.setEmployeeId(101);
        e.setEmployeeName("Rahul");

        e.setSalary(50000);      // valid
        e.displayDetails();
        System.out.println();

        e.setSalary(-2000);      // invalid (negative)
        e.setSalary(1500000);    // invalid (too large)
        System.out.println();

        System.out.println("Salary after invalid attempts:");
        e.displayDetails();
    }
}
```

OUTPUT:-

```
Accepted: Salary set to 50000.0
Employee ID  : 101
Employee Name: Rahul
Salary       : 50000.0

Rejected: Salary cannot be negative (-2000.0)
Rejected: Salary cannot exceed 1,000,000 (1500000.0)

Salary after invalid attempts:
Employee ID  : 101
Employee Name: Rahul
Salary       : 50000.0
```

Q4. Animal -> Dog

Concept: Single inheritance

Hierarchy / Diagram:

```
Animal
  |
  v
 Dog
```

Program:

```
class Animal {
    void eat()   { System.out.println("Animal is eating..."); }
    void sleep() { System.out.println("Animal is sleeping..."); }
}

class Dog extends Animal {
    void bark()  { System.out.println("Dog is barking..."); }
}

class Main {
    public static void main(String[] args) {
        Dog d = new Dog();
        d.eat();     // from Animal
        d.sleep();   // from Animal
        d.bark();    // from Dog
    }
}
```

OUTPUT:-

```
Animal is eating...
Animal is sleeping...
Dog is barking...
```

Q5. Person -> Student

Concept: single inheritance and code reuse

Hierarchy / Diagram:

```
Person
  |
  v
Student
```

Program:

```
class Person {
    String name;
    int age;

    Person(String name, int age) {
        this.name = name;
        this.age = age;
    }

    void displayPerson() {
        System.out.println("Name: " + name);
        System.out.println("Age : " + age);
    }
}

class Student extends Person {
    int rollNo;
    String course;

    Student(String name, int age, int rollNo, String course) {
        super(name, age);          // reuse Person's constructor
        this.rollNo = rollNo;
        this.course = course;
    }

    void displayStudent() {
        displayPerson();           // reuse Person's method
        System.out.println("Roll No: " + rollNo);
        System.out.println("Course : " + course);
    }
}

class Main {
    public static void main(String[] args) {
        Student s = new Student("Varnika", 18, 101, "BCA");
        s.displayStudent();
    }
}
```

OUTPUT:-

```
Name: Varnika
```

```
Age : 18
Roll No: 101
Course : BCA
```

Q6. Employee -> Manager with Encapsulation

Concept: encapsulation + inheritance

Hierarchy / Diagram:

```
Employee (private name, salary)
  |
  v
Manager (department)
```

Program:

```
class Employee {
    private String name;
    private double salary;

    public String getName() { return name; }
    public void setName(String name) { this.name = name; }

    public double getSalary() { return salary; }
    public void setSalary(double salary) { this.salary = salary; }
}

class Manager extends Employee {
    private String department;

    public String getDepartment() { return department; }
    public void setDepartment(String department) { this.department = department; }

    void displayManager() {
        // Employee's private fields are accessed through inherited getters
        System.out.println("Name      : " + getName());
        System.out.println("Salary    : " + getSalary());
        System.out.println("Department: " + department);
    }
}

class Main {
    public static void main(String[] args) {
        Manager m = new Manager();
        m.setName("Amit");          // inherited setter
        m.setSalary(75000);         // inherited setter
        m.setDepartment("Sales");
        m.displayManager();
    }
}
```

OUTPUT:-

```
Name      : Amit
Salary    : 75000.0
Department: Sales
```

Q7. Animal -> Dog -> Puppy

Concept: multilevel inheritance

Hierarchy / Diagram:

```
Animal   (eat)
  |
  v
 Dog     (bark)
  |
  v
Puppy    (play)
```

Program:

```
class Animal {
    void eat()  { System.out.println("Animal is eating..."); }
}
class Dog extends Animal {
```

```
    void bark() { System.out.println("Dog is barking..."); }
}

class Puppy extends Dog {
    void play() { System.out.println("Puppy is playing..."); }
}

class Main {
    public static void main(String[] args) {
        Puppy p = new Puppy();
        p.eat();    // Animal
        p.bark();   // Dog
        p.play();   // Puppy
    }
}
```

OUTPUT:-

```
Animal is eating...
Dog is barking...
Puppy is playing...
```

Q8. Person -> Employee -> Manager

Concept: multilevel inheritance

Hierarchy / Diagram:

```
Person    (name, displayName)
  |
  v
Employee  (employeeId, displayEmployee)
  |
  v
Manager   (department, displayManager)
```

Members of each class:

Class	Members
Person	name, displayName()
Employee	employeeId, displayEmployee() (+ inherited from Person)
Manager	department, displayManager() (+ inherited from Employee and Person)

Program:

```
class Person {
    String name;

    Person(String name) { this.name = name; }

    void displayName() { System.out.println("Name       : " + name); }
}

class Employee extends Person {
    int employeeId;

    Employee(String name, int employeeId) {
        super(name);
        this.employeeId = employeeId;
    }

    void displayEmployee() { System.out.println("Employee ID: " + employeeId); }
}

class Manager extends Employee {
    String department;

    Manager(String name, int employeeId, String department) {
        super(name, employeeId);
        this.department = department;
    }

    void displayManager() { System.out.println("Department : " + department); }
}
class Main {
```

```
    public static void main(String[] args) {
        Manager m = new Manager("Amit", 501, "Sales");
        m.displayName();       // belongs to Person
        m.displayEmployee();   // belongs to Employee
        m.displayManager();    // belongs to Manager
    }
}
```

OUTPUT:-

```
Name       : Amit
Employee ID: 501
Department : Sales
```

Q9. Vehicle -> Car -> ElectricCar

Concept: multilevel inheritance and inherited behavior

Hierarchy / Diagram:

```
Vehicle      (start, stop)
  |
  v
 Car         (drive)
  |
  v
ElectricCar  (chargeBattery)
```

Program:

```
class Vehicle {
    void start() { System.out.println("Vehicle started."); }
    void stop()  { System.out.println("Vehicle stopped."); }
}

class Car extends Vehicle {
    void drive() { System.out.println("Car is being driven."); }
}

class ElectricCar extends Car {
    void chargeBattery() { System.out.println("Electric car battery is charging."); }
}

class Main {
    public static void main(String[] args) {
        ElectricCar ec = new ElectricCar();
        ec.start();           // Vehicle
        ec.drive();           // Car
        ec.chargeBattery();   // ElectricCar
        ec.stop();            // Vehicle
    }
}
```

OUTPUT:-

```
Vehicle started.
Car is being driven.
Electric car battery is charging.
Vehicle stopped.
```

Q10. Animal -> Dog and Cat

Concept: hierarchical inheritance

Hierarchy / Diagram:

```
      Animal (eat)
      /    \
     v      v
   Dog      Cat
 (bark)    (meow)
```

Program:

```
class Animal {
    void eat() { System.out.println("Animal is eating..."); }
}

class Dog extends Animal {
```

```
    void bark() { System.out.println("Dog is barking..."); }
}

class Cat extends Animal {
    void meow() { System.out.println("Cat is meowing..."); }
}

class Main {
    public static void main(String[] args) {
        Dog d = new Dog();
        Cat c = new Cat();

        System.out.println("--- Dog ---");
        d.eat();
        d.bark();

        System.out.println("--- Cat ---");
        c.eat();
        c.meow();
    }
}
```

OUTPUT:-

```
--- Dog ---
Animal is eating...
Dog is barking...
--- Cat ---
Animal is eating...
Cat is meowing...
```

Q11. Person -> Student and Teacher

Concept: hierarchical inheritance

Hierarchy / Diagram:

```
      Person (name, displayName)
      /     \
     v       v
Student    Teacher
(course,   (subject,
 study)     teach)
```

Program:

```
class Person {
    String name;

    Person(String name) { this.name = name; }

    void displayName() { System.out.println("Name   : " + name); }
}

class Student extends Person {
    String course;

    Student(String name, String course) {
        super(name);
        this.course = course;
    }

    void study() { System.out.println(name + " is studying " + course + "."); }
}

class Teacher extends Person {
    String subject;

    Teacher(String name, String subject) {
        super(name);
        this.subject = subject;
    }

    void teach() { System.out.println(name + " is teaching " + subject + "."); }
}

class Main {
    public static void main(String[] args) {
```

```
        Student s = new Student("Varnika", "BCA");
        Teacher t = new Teacher("Dr. Sharma", "Java");

        System.out.println("--- Student ---");
        s.displayName();
        System.out.println("Course : " + s.course);
        s.study();

        System.out.println("--- Teacher ---");
        t.displayName();
        System.out.println("Subject: " + t.subject);
        t.teach();
    }
}
```

OUTPUT:-

```
--- Student ---
Name   : Varnika
Course : BCA
Varnika is studying BCA.
--- Teacher ---
Name   : Dr. Sharma
Subject: Java
Dr. Sharma is teaching Java.
```

Q12. Employee -> Developer and Manager

Concept: hierarchical inheritance

Hierarchy / Diagram:

```
         Employee (name, id, displayEmployee)
          /      \
         v        v
 Developer      Manager
 (programmingLanguage,   (department,
  writeCode)              conductMeeting)
```

Members of each class:

Class	Inherited members	Child-specific members
Developer	name, employeeId, displayEmployee()	programmingLanguage, writeCode()
Manager	name, employeeId, displayEmployee()	department, conductMeeting()

Program:

```
class Employee {
    String employeeName;
    int employeeId;

    Employee(String employeeName, int employeeId) {
        this.employeeName = employeeName;
        this.employeeId = employeeId;
    }

    void displayEmployee() {
        System.out.println("Name: " + employeeName + " | ID: " + employeeId);
    }
}

class Developer extends Employee {
    String programmingLanguage;

    Developer(String name, int id, String programmingLanguage) {
        super(name, id);
        this.programmingLanguage = programmingLanguage;
    }

    void writeCode() {
        System.out.println(employeeName + " is writing code in " + programmingLanguage + ".");
    }
}

class Manager extends Employee {
    String department;
```

```
    Manager(String name, int id, String department) {
        super(name, id);
        this.department = department;
    }

    void conductMeeting() {
        System.out.println(employeeName + " is conducting a meeting of " + department + " department.");
    }
}

class Main {
    public static void main(String[] args) {
        Developer d = new Developer("Rahul", 201, "Java");
        Manager m = new Manager("Amit", 301, "Sales");

        System.out.println("--- Developer ---");
        d.displayEmployee();   // inherited
        d.writeCode();         // child-specific

        System.out.println("--- Manager ---");
        m.displayEmployee();   // inherited
        m.conductMeeting();    // child-specific
    }
}
```

OUTPUT:-

```
--- Developer ---
Name: Rahul | ID: 201
Rahul is writing code in Java.
--- Manager ---
Name: Amit | ID: 301
Amit is conducting a meeting of Sales department.
```

Q13. Animal Sound Overriding

Concept: method overriding

Program:

```
class Animal {
    void makeSound() { System.out.println("Animal makes a sound."); }
}

class Dog extends Animal {
    @Override
    void makeSound() { System.out.println("Dog says: Woof! Woof!"); }
}

class Cat extends Animal {
    @Override
    void makeSound() { System.out.println("Cat says: Meow! Meow!"); }
}

class Main {
    public static void main(String[] args) {
        Animal a = new Animal();
        Dog d = new Dog();
        Cat c = new Cat();
        Animal ref = new Dog();   // parent reference, child object

        a.makeSound();
        d.makeSound();
        c.makeSound();
        ref.makeSound();          // child's version runs
    }
}
```

OUTPUT:-

```
Animal makes a sound.
Dog says: Woof! Woof!
Cat says: Meow! Meow!
Dog says: Woof! Woof!
```

Q14. Employee Salary Calculation

Concept: inheritance + method overriding

Program:

```
class Employee {
    double basicSalary;

    Employee(double basicSalary) { this.basicSalary = basicSalary; }

    // Employee: Basic + 10% allowance
    double calculateSalary() {
        return basicSalary + (basicSalary * 0.10);
    }
}

class Manager extends Employee {
    Manager(double basicSalary) { super(basicSalary); }

    // Manager: Basic + 30% allowance + fixed bonus 10000
    @Override
    double calculateSalary() {
        return basicSalary + (basicSalary * 0.30) + 10000;
    }
}

class Main {
    public static void main(String[] args) {
        Employee e = new Employee(40000);
        Manager m = new Manager(40000);
        Employee ref = new Manager(60000);   // parent reference, Manager object

        System.out.println("Employee salary          : " + e.calculateSalary());
        System.out.println("Manager salary           : " + m.calculateSalary());
        System.out.println("Employee ref -> Manager  : " + ref.calculateSalary()
                + "  (Manager's method executed)");
    }
}
```

OUTPUT:-

```
Employee salary          : 44000.0
Manager salary           : 62000.0
Employee ref -> Manager  : 88000.0  (Manager's method executed)
```

Q15. Shape Area Overriding

Concept: same method name with different child implementations

Program:

```
class Shape {
    double calculateArea() {
        return 0;
    }
}

class Circle extends Shape {
    double radius;

    Circle(double radius) { this.radius = radius; }

    @Override
    double calculateArea() {
        return Math.PI * radius * radius;
    }
}

class Rectangle extends Shape {
    double length, breadth;

    Rectangle(double length, double breadth) {
        this.length = length;
        this.breadth = breadth;
    }

    @Override
    double calculateArea() {
```

```
        return length * breadth;
    }
}

class Main {
    public static void main(String[] args) {
        Shape c = new Circle(7);
        Shape r = new Rectangle(10, 5);

        System.out.printf("Area of Circle (r = 7)         : %.2f%n", c.calculateArea());
        System.out.printf("Area of Rectangle (10 x 5)     : %.2f%n", r.calculateArea());
    }
}
```

OUTPUT:-

```
Area of Circle (r = 7)         : 153.94
Area of Rectangle (10 x 5)     : 50.00
```

Q16. Camera + MusicPlayer -> Smartphone

Concept: multiple inheritance using interfaces

Hierarchy / Diagram:

```
Camera       MusicPlayer
   \           /
    \         /
     v       v
     Smartphone
```

Program:

```
interface Camera {
    void takePhoto();
}

interface MusicPlayer {
    void playMusic();
}

class Smartphone implements Camera, MusicPlayer {
    @Override
    public void takePhoto() { System.out.println("Smartphone is taking a photo."); }

    @Override
    public void playMusic() { System.out.println("Smartphone is playing music."); }
}

class Main {
    public static void main(String[] args) {
        Smartphone s = new Smartphone();
        s.takePhoto();
        s.playMusic();
    }
}
```

OUTPUT:-

```
Smartphone is taking a photo.
Smartphone is playing music.
```

Q17. Printer + Scanner -> MultiFunctionMachine

Concept: multiple interfaces + encapsulation

Hierarchy / Diagram:

```
Printer        Scanner
   \             /
    \           /
     v         v
  MultiFunctionMachine
```

Program:

```
interface Printer {
    void print();
}
```

```
interface Scanner {
    void scan();
}

class MultiFunctionMachine implements Printer, Scanner {
    private String machineName;
    private int machineId;

    public String getMachineName() { return machineName; }
    public void setMachineName(String machineName) { this.machineName = machineName; }

    public int getMachineId() { return machineId; }
    public void setMachineId(int machineId) { this.machineId = machineId; }

    @Override
    public void print() { System.out.println(machineName + " is printing a document."); }

    @Override
    public void scan()  { System.out.println(machineName + " is scanning a document."); }
}

class Main {
    public static void main(String[] args) {
        MultiFunctionMachine m = new MultiFunctionMachine();
        m.setMachineName("HP LaserJet");
        m.setMachineId(7001);

        System.out.println("Machine Name: " + m.getMachineName());
        System.out.println("Machine ID  : " + m.getMachineId());
        m.print();
        m.scan();
    }
}
```

OUTPUT:-

```
Machine Name: HP LaserJet
Machine ID  : 7001
HP LaserJet is printing a document.
HP LaserJet is scanning a document.
```

Q18. Vehicle -> Car -> ElectricCar + Electric

Concept: multilevel inheritance + interface implementation + hybrid structure

Hierarchy / Diagram:

```
Vehicle  (start)
  |
  v
 Car     (drive)           <<interface>> Electric (chargeBattery)
   \                              /
    \                            /
     v                          v
          ElectricCar
```

Program:

```
class Vehicle {
    void start() { System.out.println("Vehicle started."); }
}

class Car extends Vehicle {
    void drive() { System.out.println("Car is being driven."); }
}

interface Electric {
    void chargeBattery();
}

// extends (multilevel) + implements (interface) = hybrid structure
class ElectricCar extends Car implements Electric {
    @Override
    public void chargeBattery() { System.out.println("Electric car battery is charging."); }
}

class Main {
    public static void main(String[] args) {
```

```
        ElectricCar ec = new ElectricCar();
        ec.start();           // Vehicle
        ec.drive();           // Car
        ec.chargeBattery();   // Electric
    }
}
```

OUTPUT:-

```
Vehicle started.
Car is being driven.
Electric car battery is charging.
```

Q19. Employee -> Developer + Programmer + Researcher

Concept: encapsulation, inheritance, multiple interfaces, hybrid structure

Hierarchy / Diagram:

```
   Employee      <<interface>>    <<interface>>
  (name, id)       Programmer       Researcher
       \               |               /
        \              |              /
         v             v             v
                   Developer
    (extends Employee, implements Programmer, Researcher)
```

Program:

```
class Employee {
    private String name;
    private int employeeId;

    public String getName() { return name; }
    public void setName(String name) { this.name = name; }

    public int getEmployeeId() { return employeeId; }
    public void setEmployeeId(int employeeId) { this.employeeId = employeeId; }

    void displayEmployee() {
        System.out.println("Name       : " + name);
        System.out.println("Employee ID: " + employeeId);
    }
}

interface Programmer {
    void writeCode();
}

interface Researcher {
    void conductResearch();
}

class Developer extends Employee implements Programmer, Researcher {
    @Override
    public void writeCode() {
        System.out.println(getName() + " is writing code.");
    }

    @Override
    public void conductResearch() {
        System.out.println(getName() + " is conducting research on new technologies.");
    }
}

class Main {
    public static void main(String[] args) {
        Developer d = new Developer();
        d.setName("Rahul");
        d.setEmployeeId(201);

        d.displayEmployee();   // from Employee
        d.writeCode();         // from Programmer
        d.conductResearch();   // from Researcher
    }
}
```

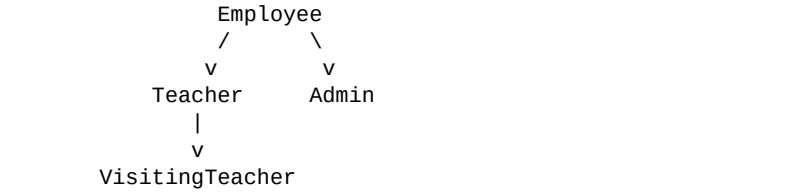
OUTPUT:-

```
Name       : Rahul
Employee ID: 201
Rahul is writing code.
Rahul is conducting research on new technologies.
```

Q20. University Employee System - Integrated OOP

Concept: all major OOP concepts together

Hierarchy / Diagram:




```
   Researcher, Mentor (interfaces)  --implemented by-->  Teacher

Employee -> Teacher -> VisitingTeacher        : Multilevel
Employee -> Teacher and Employee -> Admin     : Hierarchical
Teacher extends Employee + implements 2 interfaces : Hybrid
```

Concepts covered:

Concept	Where it is shown
Encapsulation / Getter-Setter	Employee: private id, name, salary with getters/setters
Single inheritance	Admin extends Employee
Multilevel inheritance	Employee -> Teacher -> VisitingTeacher
Hierarchical inheritance	Teacher and Admin both extend Employee
Interface / Multiple inheritance via interfaces	Researcher, Mentor interfaces; Teacher implements both
Hybrid inheritance	Teacher extends Employee implements Researcher, Mentor
Method overriding + @Override	calculateSalary(), displayDetails(), conductResearch()
super keyword	super(...) in constructors, super.calculateSalary(), super.displayDetails()
Multiple objects	Teacher, VisitingTeacher, Admin objects in an array

Program:

```
interface Researcher {
    void conductResearch();
}

interface Mentor {                        // extra interface to show multiple interfaces
    void mentorStudents();
}

class Employee {
    private int employeeId;
    private String employeeName;
    private double salary;

    Employee(int employeeId, String employeeName, double salary) {
        this.employeeId = employeeId;
        this.employeeName = employeeName;
        this.salary = salary;
    }

    public int getEmployeeId() { return employeeId; }
    public void setEmployeeId(int employeeId) { this.employeeId = employeeId; }

    public String getEmployeeName() { return employeeName; }
    public void setEmployeeName(String employeeName) { this.employeeName = employeeName; }

    public double getSalary() { return salary; }
    public void setSalary(double salary) {
        if (salary >= 0) this.salary = salary;
    }
```

```
    void displayDetails() {
        System.out.println("ID    : " + employeeId);
        System.out.println("Name  : " + employeeName);
        System.out.println("Salary: " + calculateSalary());
    }

    double calculateSalary() {
        return salary;                              // base salary
    }
}

class Teacher extends Employee implements Researcher, Mentor {
    String subject;

    Teacher(int id, String name, double salary, String subject) {
        super(id, name, salary);
        this.subject = subject;
    }

    void teach() { System.out.println(getEmployeeName() + " is teaching " + subject + "."); }

    @Override
    public void conductResearch() {
        System.out.println(getEmployeeName() + " is conducting research in " + subject + ".");
    }

    @Override
    public void mentorStudents() {
        System.out.println(getEmployeeName() + " is mentoring students.");
    }

    @Override
    double calculateSalary() {
        return super.calculateSalary() + 15000;      // base + teaching allowance
    }

    @Override
    void displayDetails() {
        super.displayDetails();
        System.out.println("Subject: " + subject);
    }
}

class VisitingTeacher extends Teacher {
    int hoursWorked;
    double ratePerHour = 1000;

    VisitingTeacher(int id, String name, String subject, int hoursWorked) {
        super(id, name, 0, subject);
        this.hoursWorked = hoursWorked;
    }

    @Override
    double calculateSalary() {
        return hoursWorked * ratePerHour;            // paid per hour
    }

    @Override
    void displayDetails() {
        super.displayDetails();
        System.out.println("Hours Worked: " + hoursWorked);
    }
}

class Admin extends Employee {
    String department;

    Admin(int id, String name, double salary, String department) {
        super(id, name, salary);
        this.department = department;
    }

    void manageDepartment() {
        System.out.println(getEmployeeName() + " is managing the " + department + " department.");
    }
    @Override
```

```
    double calculateSalary() {
        return super.calculateSalary() + 8000;       // base + admin allowance
    }

    @Override
    void displayDetails() {
        super.displayDetails();
        System.out.println("Department: " + department);
    }
}

class Main {
    public static void main(String[] args) {
        Teacher t = new Teacher(1, "Dr. Sharma", 60000, "Java");
        VisitingTeacher vt = new VisitingTeacher(2, "Prof. Verma", "Python", 40);
        Admin a = new Admin(3, "Mr. Gupta", 45000, "Accounts");

        System.out.println("===== Teacher =====");
        t.displayDetails();
        t.teach();
        t.conductResearch();
        t.mentorStudents();

        System.out.println("\n===== Visiting Teacher =====");
        vt.displayDetails();
        vt.teach();
        vt.conductResearch();

        System.out.println("\n===== Admin =====");
        a.displayDetails();
        a.manageDepartment();

        // Polymorphism: Employee reference -> different child objects
        System.out.println("\n===== Salary Report (Polymorphism) =====");
        Employee[] staff = { t, vt, a };
        for (Employee e : staff) {
            System.out.println(e.getEmployeeName() + " -> " + e.calculateSalary());
        }
    }
}
```

OUTPUT:-

```
===== Teacher =====
ID    : 1
Name  : Dr. Sharma
Salary: 75000.0
Subject: Java
Dr. Sharma is teaching Java.
Dr. Sharma is conducting research in Java.
Dr. Sharma is mentoring students.

===== Visiting Teacher =====
ID    : 2
Name  : Prof. Verma
Salary: 40000.0
Subject: Python
Hours Worked: 40
Prof. Verma is teaching Python.
Prof. Verma is conducting research in Python.

===== Admin =====
ID    : 3
Name  : Mr. Gupta
Salary: 53000.0
Department: Accounts
Mr. Gupta is managing the Accounts department.

===== Salary Report (Polymorphism) =====
Dr. Sharma -> 75000.0
Prof. Verma -> 40000.0
Mr. Gupta -> 53000.0
```

Note: Mentor is a small extra interface added only to show multiple interfaces (Researcher + Mentor) implemented by one class.